

Session-7

Question1: Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.

Ans:

```
CREATE TABLE Restaurant (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    location VARCHAR(100) NOT NULL,  
    cuisine VARCHAR(50)  
);
```

Question2: Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).

Ans:

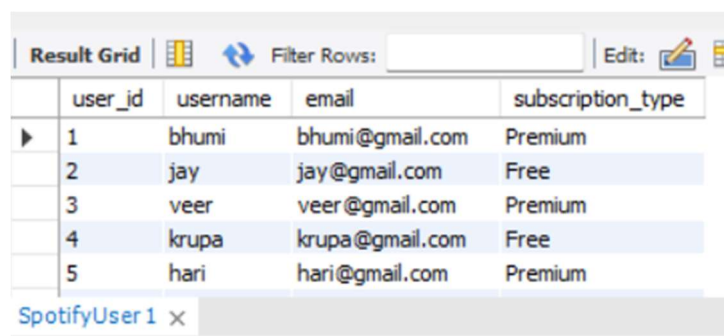
```
CREATE TABLE FoodOrder (  
    order_id INT PRIMARY KEY,  
    restaurant_id INT,  
    user_id INT,  
    order_total DECIMAL(10,2),  
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)  
);
```

Question3: Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.

Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

Ans:

```
CREATE TABLE SpotifyUser (  
    user_id INT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE,  
    email VARCHAR(100) NOT NULL,  
    subscription_type VARCHAR(30)  
);  
  
INSERT INTO SpotifyUser (user_id, username, email, subscription_type)VALUES  
(1, 'bhumi', 'bhumi@gmail.com', 'Premium'),  
(2, 'jay', 'jay@gmail.com', 'Free'),  
(3, 'veer', 'veer@gmail.com', 'Premium'),  
(4, 'krupa', 'krupa@gmail.com', 'Free'),  
(5, 'hari', 'hari@gmail.com', 'Premium'),  
(6, 'meera', 'meera@gmail.com', 'Free');  
  
SELECT * FROM SpotifyUser;
```



The screenshot shows a database interface with a 'Result Grid' tab. It displays the contents of the SpotifyUser table. The table has four columns: user_id, username, email, and subscription_type. There are five rows of data visible. The interface includes a 'Filter Rows' search bar and an 'Edit' button. A tab labeled 'SpotifyUser1' is visible at the bottom.

	user_id	username	email	subscription_type
▶	1	bhumi	bhumi@gmail.com	Premium
	2	jay	jay@gmail.com	Free
	3	veer	veer@gmail.com	Premium
	4	krupa	krupa@gmail.com	Free
	5	hari	hari@gmail.com	Premium

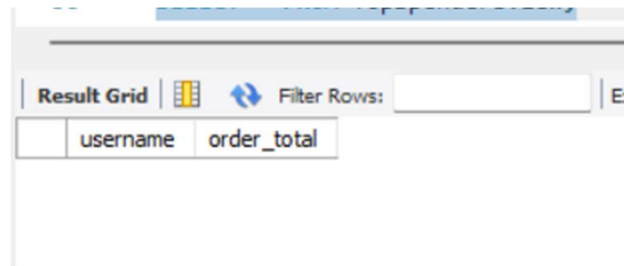
Question4: Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

Ans:

```
CREATE VIEW TopSpendersView AS SELECT SpotifyUser.username,  
FoodOrder.order_total FROM FoodOrder
```

```
INNER JOIN SpotifyUser ON FoodOrder.user_id = SpotifyUser.user_id WHERE  
FoodOrder.order_total > 1000;
```

```
SELECT * FROM TopSpendersView;
```



Question5: Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending.

Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.

Ans:

```
SELECT username, order_total FROM TopSpendersView WHERE order_total > 2000  
ORDER BY order_total DESC;
```